RAJALAKSHMI ENGINEERING COLLEGE

Laxmi Gayathiri S 251801157-2025-AIDS L2

Started on	Thursday, 17 September 2026, 10:14 AM
State	Finished
Completed on	Wednesday, 30 September 2026, 11:54 AM
Time taken	13 days 1 hour
Marks	1.00/1.00
Grade	**10.00** out of 10.00 (**100**%)

Question 1 | Correct Mark 1.00 out of 1.00

Problem Statement

Given an array of 1s and 0s this has all 1s first followed by all 0s. Aim is to find the number of 0s. Write a program using Divide and Conquer to Count the number of zeroes in the given array.

Input Format

First Line Contains Integer m – Size of array

Next m lines Contains m numbers – Elements of an array

Output Format

First Line Contains Integer – Number of zeroes present in the given array.

Answer: (penalty regime: 0 %)

```c
#include <stdio.h>

// Function to find the index of the first occurrence of 0
int findFirstZero(int arr[], int low, int high) {
    if (high >= low) {
        int mid = low + (high - low) / 2;

        // Check if mid is the first occurrence of 0
        if ((mid == 0 || arr[mid - 1] == 1) && arr[mid] == 0) {
            return mid;
        }

        // If the element is 1, the first 0 must be in the right half
        if (arr[mid] == 1) {
            return findFirstZero(arr, mid + 1, high);
        }

        // If the element is 0 but not the first 0, it must be in the left half
        return findFirstZero(arr, low, mid - 1);
    }

    // Return -1 if no 0 is found in the array
    return -1;
}

int main() {
    int m;

    // Read the size of the array
    if (scanf("%d", &m) != 1 || m <= 0) {
        printf("0\n");
        return 0;
    }

    int arr[m];

    // Read the m elements line by line
    for (int i = 0; i < m; i++) {
        scanf("%d", &arr[i]);
    }

    // Find the index of the first zero
    int firstZeroIndex = findFirstZero(arr, 0, m - 1);

    // If no 0 is found, print 0. Otherwise, subtract the index from total size.
    if (firstZeroIndex == -1) {
        printf("0\n");
    } else {
        printf("%d\n", m - firstZeroIndex);
    }

    return 0;
```

	Input	Expected	Got	
✔	5 1 1 1 0 0	2	2	✔
✔	10 1 1 1 1 1 1 1 1 1 1	0	0	✔
✔	8 0 0 0 0 0 0 0 0	8	8	✔
✔	17 1 1 1 1 1 1 1 1 1 1 1 1 1 1 0 0	2	2	✔

Passed all tests! ✔

Correct

Marks for this submission: 1.00/1.00.

Back to Course

RAJALAKSHMI ENGINEERING COLLEGE

Laxmi Gayathiri S 251801157-2025-AIDS

L2

Started on	Thursday, 17 September 2026, 10:18 AM
State	Finished
Completed on	Wednesday, 30 September 2026, 3:13 PM
Time taken	13 days 4 hours
Marks	1.00/1.00
Grade	**10.00** out of 10.00 (**100**%)

Question 1 | Correct Mark 1.00 out of 1.00

Given an array `nums` of size `n`, return *the majority element*.

The majority element is the element that appears more than `⌊n / 2⌋` times. You may assume that the majority element always exists in the array.

Example 1:

```
Input: nums = [3,2,3]
Output: 3
```

Example 2:

```
Input: nums = [2,2,1,1,1,2,2]
Output: 2
```

Constraints:

- `n == nums.length`
- `1 <= n <= 5 * 10^4`
- `-2^31 <= nums[i] <= 2^31 - 1`

For example:

Input	Result
3 3 2 3	3
7 2 2 1 1 1 2 2	2

Answer: (penalty regime: 0 %)

```c
#include <stdio.h>

// Function to find the majority element using Boyer-Moore Voting Algorithm
int findMajorityElement(int arr[], int n) {
    int candidate = arr[0];
    int count = 0;

    for (int i = 0; i < n; i++) {
        if (count == 0) {
            candidate = arr[i];
        }

        if (arr[i] == candidate) {
            count++;
        } else {
            count--;
        }
    }

    return candidate;
}

int main() {
    int n;

    // Read the size of the array
    if (scanf("%d", &n) != 1 || n <= 0) {
        return 0;
    }

    int arr[n];

    // Read the array elements
    for (int i = 0; i < n; i++) {
```

```
        scanf("%d", &arr[i]);
    }

    // Find and print the majority element
    int majority = findMajorityElement(arr, n);
    printf("%d\n", majority);

    return 0;
}
```

	Input	Expected	Got	
✔	3 3 2 3	3	3	✔

Passed all tests! ✔

Correct

Marks for this submission: 1.00/1.00.

Back to Course

RAJALAKSHMI ENGINEERING COLLEGE

Laxmi Gayathiri S 251801157-2025-AIDS

L2

Started on	Thursday, 17 September 2026, 10:19 AM
State	Finished
Completed on	Wednesday, 30 September 2026, 3:14 PM
Time taken	13 days 4 hours
Marks	1.00/1.00
Grade	**10.00** out of 10.00 (**100**%)

Question 1 | Correct Mark 1.00 out of 1.00

Problem Statement:

Given a sorted array and a value x, the floor of x is the largest element in array smaller than or equal to x. Write divide and conquer algorithm to find floor of x.

Input Format

First Line Contains Integer n – Size of array

Next n lines Contains n numbers – Elements of an array

Last Line Contains Integer x – Value for x

Output Format

First Line Contains Integer – Floor value for x

Answer: (penalty regime: 0 %)

```c
#include <stdio.h>

// Function to find the floor of x using Divide and Conquer
int findFloor(int arr[], int n, int x) {
    int low = 0, high = n - 1;
    int floorValue = -1; // Default if no floor exists (all elements > x)

    while (low <= high) {
        int mid = low + (high - low) / 2;

        // If x is exactly equal to mid element, mid is the floor
        if (arr[mid] == x) {
            return arr[mid];
        }

        // If arr[mid] is smaller than x, it's a potential floor.
        // Move to the right half to look for a larger valid floor.
        if (arr[mid] < x) {
            floorValue = arr[mid];
            low = mid + 1;
        }
        // If arr[mid] is greater than x, look in the left half
        else {
            high = mid - 1;
        }
    }

    return floorValue;
}

int main() {
    int n, x;

    // Read the size of the array
    if (scanf("%d", &n) != 1 || n <= 0) {
        return 0;
    }

    int arr[n];

    // Read the n elements line by line
    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }

    // Read the target value x
    if (scanf("%d", &x) != 1) {
        return 0;
    }

    // Find and print the floor value
    int result = findFloor(arr, n, x);
```

	Input	Expected	Got	
✔	6 1 2 8 10 12 19 5	2	2	✔
✔	5 10 22 85 108 129 100	85	85	✔
✔	7 3 5 7 9 11 13 15 10	9	9	✔

Passed all tests! ✔

Correct

Marks for this submission: 1.00/1.00.

Back to Course

RAJALAKSHMI ENGINEERING COLLEGE

Laxmi Gayathiri S 251801157-2025-AIDS

L2

Started on	Thursday, 17 September 2026, 10:30 AM
State	Finished
Completed on	Wednesday, 30 September 2026, 3:14 PM
Time taken	13 days 4 hours
Marks	1.00/1.00
Grade	**10.00** out of 10.00 (**100**%)

Question 1 | Correct Mark 1.00 out of 1.00

Problem Statement:

Given a sorted array of integers say arr[] and a number x. Write a recursive program using divide and conquer strategy to check if there exist two elements in the array whose sum = x. If there exist such two elements then return the numbers, otherwise print as "No".

Note: Write a Divide and Conquer Solution

Input Format

First Line Contains Integer n – Size of array

Next n lines Contains n numbers – Elements of an array

Last Line Contains Integer x – Sum Value

Output Format

First Line Contains Integer – Element1

Second Line Contains Integer – Element2 (Element 1 and Elements 2 together sums to value "x")

Answer: (penalty regime: 0 %)

```c
#include <stdio.h>
#include <stdlib.h>

// Recursive Divide and Conquer function to find the pair
int findPairSum(int arr[], int low, int high, int x, int *element1, int *element2) {
    // Base Case: If the pointers cross, no such pair exists
    if (low >= high) {
        return 0;
    }

    int current_sum = arr[low] + arr[high];

    // If target sum is found, store the elements and return success
    if (current_sum == x) {
        *element1 = arr[low];
        *element2 = arr[high];
        return 1;
    }

    // If current sum is smaller, discard the smaller element (increment low)
    if (current_sum < x) {
        return findPairSum(arr, low + 1, high, x, element1, element2);
    }
    // If current sum is larger, discard the larger element (decrement high)
    else {
        return findPairSum(arr, low, high - 1, x, element1, element2);
    }
}

int main() {
    int n;

    // Read the size of the array
    if (scanf("%d", &n) != 1) return 0;

    int *arr = (int *)malloc(n * sizeof(int));
    if (arr == NULL) return 1;

    // Read the array elements
    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }

    int x;
    // Read the target sum value
    scanf("%d", &x);

    int element1 = 0, element2 = 0;

    // Invoke the recursive function
    if (findPairSum(arr, 0, n - 1, x, &element1, &element2)) {
        printf("%d\n", element1);
```

	Input	Expected	Got	
✔	4 2 4 8 10 14	4 10	4 10	✔
✔	5 2 4 6 8 10 100	No	No	✔

Passed all tests! ✔

Correct

Marks for this submission: 1.00/1.00.

Back to Course

RAJALAKSHMI ENGINEERING COLLEGE

Laxmi Gayathiri S 251801157-2025-AIDS

L2

Started on	Thursday, 17 September 2026, 10:39 AM
State	Finished
Completed on	Wednesday, 30 September 2026, 3:15 PM
Time taken	13 days 4 hours
Marks	1.00/1.00
Grade	**10.00** out of 10.00 (**100**%)

Question 1 | Correct Mark 1.00 out of 1.00

Write a Program to Implement the Quick Sort Algorithm

Input Format:
The first line contains the no of elements in the list-n
The next n lines contain the elements.

Output:
Sorted list of elements

For example:

Input	Result
5 67 34 12 98 78	12 34 67 78 98

Answer:

```c
#include <stdio.h>
#include <stdlib.h>

// Function to swap two elements
void swap(int *a, int *b) {
    int temp = *a;
    *a = *b;
    *b = temp;
}

// Partition function that places the pivot element at its correct position
int partition(int arr[], int low, int high) {
    int pivot = arr[high]; // Choosing the last element as pivot
    int i = (low - 1);     // Index of smaller element

    for (int j = low; j < high; j++) {
        // If current element is smaller than or equal to pivot
        if (arr[j] <= pivot) {
            i++;
            swap(&arr[i], &arr[j]);
        }
    }
    // Swap the pivot element with the element at (i + 1)
    swap(&arr[i + 1], &arr[high]);
    return (i + 1);
}

// The main function that implements Quick Sort
void quickSort(int arr[], int low, int high) {
    if (low < high) {
        // pi is partitioning index, arr[p] is now at right place
        int pi = partition(arr, low, high);

        // Separately sort elements before partition and after partition
        quickSort(arr, low, pi - 1);
        quickSort(arr, pi + 1, high);
    }
}

int main() {
    int n;

    // Read the number of elements
    if (scanf("%d", &n) != 1) return 0;

    int *arr = (int *)malloc(n * sizeof(int));
    if (arr == NULL) return 1;

    // Read the array elements
    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }
```

	Input	Expected	Got	
✔	5 67 34 12 98 78	12 34 67 78 98	12 34 67 78 98	✔
✔	10 1 56 78 90 32 56 11 10 90 114	1 10 11 32 56 56 78 90 90 114	1 10 11 32 56 56 78 90 90 114	✔
✔	12 9 8 7 6 5 4 3 2 1 10 11 90	1 2 3 4 5 6 7 8 9 10 11 90	1 2 3 4 5 6 7 8 9 10 11 90	✔

Passed all tests! ✔

Correct

Marks for this submission: 1.00/1.00.

Back to Course